Eight weeks ago, you were building a simple calculator.

Every student here started in the same place. This page puts your Week 4 and Week 8 challenges side by side so you can see the distance you've covered, whichever path you took, and shows where Week 9 takes you.

The road so far

Week 1: a simple calculator

Two numbers, an operator, one result. This is where everyone began.

Week 4

Pure functions, map and filter, and a first taste of recursion. You built a Transaction Analyzer out of small, focused functions.

Week 8

Familiar problems at a higher bar: binary search, run-length encoding, Tower of Hanoi, sorting with counted work, Hangman. You tested your own edge cases. Some of you also took on the Performance Lab.

Week 9

Two paths, Core and Elite, and one shared last question: your own adventure.

The same idea, four weeks apart

In Week 4, recursion was a first taste: take a list nested inside lists and flatten it.

In Week 8, everyone used it as a real tool.

Week 4, Question 8

```
flatten([1, [2, [3, [4]], 5]])
# [1, 2, 3, 4, 5]

# solve the small case,
# trust the recursive call
# no loops inside flatten
```

Week 8, Questions 5 and 9

```
hanoi(3, "A", "C", "B")  # 7 moves
subsets([1, 2, 3])       # 8 results
permutations([1, 2, 3])  # 6 results

# never modify the input
# print the counts for n = 0 to 8
# notice 2^n and n!
```

Week 4 asked you to see a pattern in nested data. Week 8 asked you to build answers on the way back out of recursion, and to notice how fast the number of results grows.

What you'd have called hard then, and what you do now

Skill	Week 4	Week 8
Pure functions	Return a value and print outside, as in the two-pointer palindrome	Return a value and also report the work done, like binary search returning the index and its comparison count
One-pass thinking	Second largest, tracked by hand	Max subarray written two ways, with a check that both agree
Recursion	Flatten a nested list	Tower of Hanoi, subsets and permutations
Testing	Check against the examples given	Write at least two cases of your own, including an edge case, and test encode against decode
State in a program	Transaction Analyzer: lists of dictionaries and small pure functions	Hangman: state across many guesses, repeats and early stops, with game logic kept apart from input and output
Cost of code	Not asked	Count comparisons and swaps in bubble

Cost of code	Not asked	Count comparisons and swaps in bubble sort, and see best case against worst case
Performance Lab group	Memoize was an optional stretch	A memoize decorator with cache limits and counters, hash maps, and a written Big O analysis

What changed in the questions themselves

From "does it work?"

to "how much work does it do, and how does that grow?"

From checking the examples

to inventing your own cases and hunting for the edge that breaks it.

From trusting one answer

to proving it: brute force against single pass, encode against decode.

From writing the answer

to naming exactly what blocked you and explaining your choices.

Where this leads: two paths, one finish

Week 9 has a Core path and an Elite path. They are not a ranking. They give each of you the right amount of struggle.

Core path

- Use a stack to check balanced brackets.
- Count the work of insertion sort.
- Name your base case in every recursive function.
- Walk a grid in a spiral, and referee a game of tic-tac-toe.
- Update a shop's stock without touching the original.

Elite path

- Rebuild coin counting from the bottom up, with a table.
- Write a cache that forgets the least recently used entry.
- Find the shortest route through a network.
- Solve edit distance three ways and prove they agree.

A reminder for the climb ahead. Every question you solved this year opened a harder one: subsets grow as 2^n, permutations as n!, and some problems stop being solvable by brute force at all. The more you learn, the more you can see of what you don't know yet. That is what progress looks like, and it's a good reason to stay curious and humble.

Before Week 9 starts

Take ten minutes with a notebook and answer these honestly.

1. Which Week 4 question would you now finish in half the time, and what changed?
2. Which Week 8 idea still feels shaky: recursion, testing, tracking state, or, if you took the Performance Lab, caching and Big O? Write down the exact point where you get stuck.
3. What would you want to build in the next four weeks that you couldn't have attempted in Week 4?

Final question: your own adventure

This one isn't a stress test. You've spent weeks solving problems other people designed. Now you design one. **Suggested time: 30 to 40 minutes.**

Build a small story game where the player makes choices and the story changes. Pick any theme you like: a haunted compound, a trip through Lagos traffic, a jungle, a space station, a school where the teacher has vanished. Make it funny, scary or silly.

What a player might see

```
You wake up in a dark room.
```

```
There is a door to the NORTH and a window to the EAST.
> north
You enter a kitchen. A strange smell is coming from the fridge.
Do you OPEN the fridge or go BACK?
> open
Inside the fridge is a golden key!
```

Requirements

- At least **5 rooms** and **2 endings**, one win and one lose.
- Store your rooms in a **dictionary**, for example rooms = {"kitchen": {"text": "...", "exits": {"back": "hallway"}}}.
- Use a **loop** that runs until the player wins, loses, or types quit.
- Track at least **one item** in a **list** (a key, a map, a torch) and make it matter somewhere, such as a locked door or a safe path.
- If the player types something invalid, say so kindly and **don't crash**.
- No external libraries and no random: the story is the same every time.

Optional, if you're having fun

- Add a second item and a puzzle that needs both.
- Add a health or turn counter, for example you lose after 10 moves.
- Hide a secret room that only a curious player will find.
- Keep the logic in a small function like move(current_room, command), with print() and input() in your main loop. If your first version is all mixed together, that's fine. Make it work first.

How it's judged. Not on difficulty. Does it run without crashing? Is the story fun to read? Does at least one choice change what happens?

You started this phase building a simple calculator. Now you're building a world someone else can play in. Swap with the person next to you, play their game, and enjoy it. You've earned it.

This page only compares the Week 4 and Week 8 challenges. You have done far more than fits here. Struggling is data, not failure: write down your blockers and bring them to the next session.